CH
MapReduce & YARN

- MapReduce V1:
  - Introduction
  - Principle
  - Example: Word count
  - MRv1 : Architecture
  - Limits
- YARN: Yet Another Resource Negotiator
  - Purpose
  - MRv1 VsYARN
  - Architecture
  - Resources allocation
  - Resources planification

- Example: Let's say you have several shops that you manage around the world
  - A very large file containing ALL sales
  - Objective: Calculate total sales per shop for the current year
  - Suppose that the lines in the book have the following form:

Day	City	Product	Price
2012-01-01	London	Clothes	25.99
2012-01-01	Miami	Music	12.15
2012-01-02	NYC	Toys	3.10
2012-01-02	Miami	Clothes	50.00

[1] Intro to Hadoop and MapReduce, Udacity

- Traditional way:
  - For each entry, enter the town and the selling price
  - If you find an entry with a town already entered, group them together by summing the sales

- In a traditional calculation environment
  - we generally use Hashtables, in the form of: **<Key, Value>**
  - In our case, the **key** would be the **address** and the **value** would be the **total sales**.

2012-01-01	London	Clothes	25.99
2012-01-01	Miami	Music	12.15
2012-01-02	NYC	Toys	3.10
2012-01-02	Miami	Clothes	50.00

London	25.99
Miami	12.15
NYC	3.10

London	25.99
Miami	62.15
NYC	3.10

- Large amount of data (ex. 1TB)
  - hashtables **Problems** :
    - Memory problems ?
    - Long processing time ?

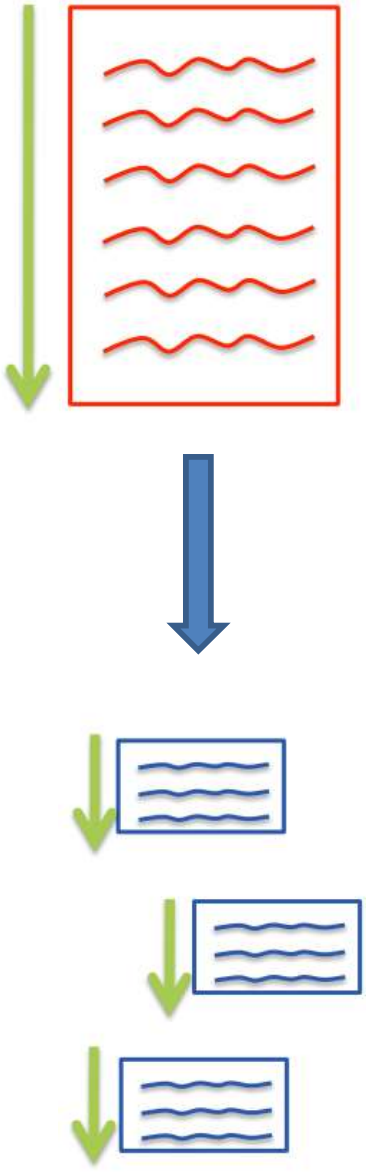
- Sequential processing of all the data can be very time-consuming
- The more shops you have, the longer it takes to add values to the table
- It is possible to run out of memory to store this table

MapReduce Paradigm

MapReduce: Idea

- Conventional algorithm :

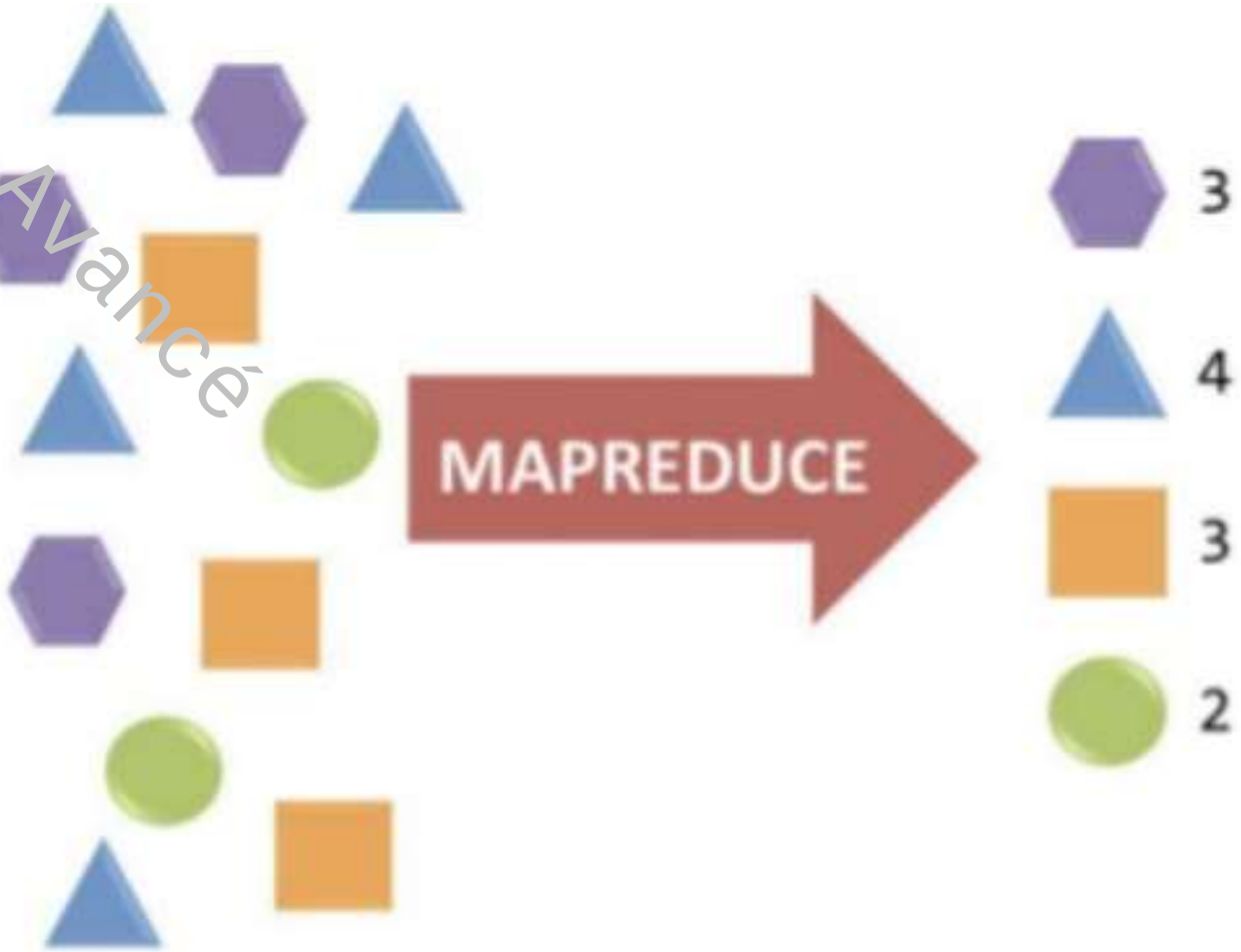
- Sequential processing
  - ✓ Very slow
  - ✓ High vertical resources
  - ✓ Not suitable for real-time applications



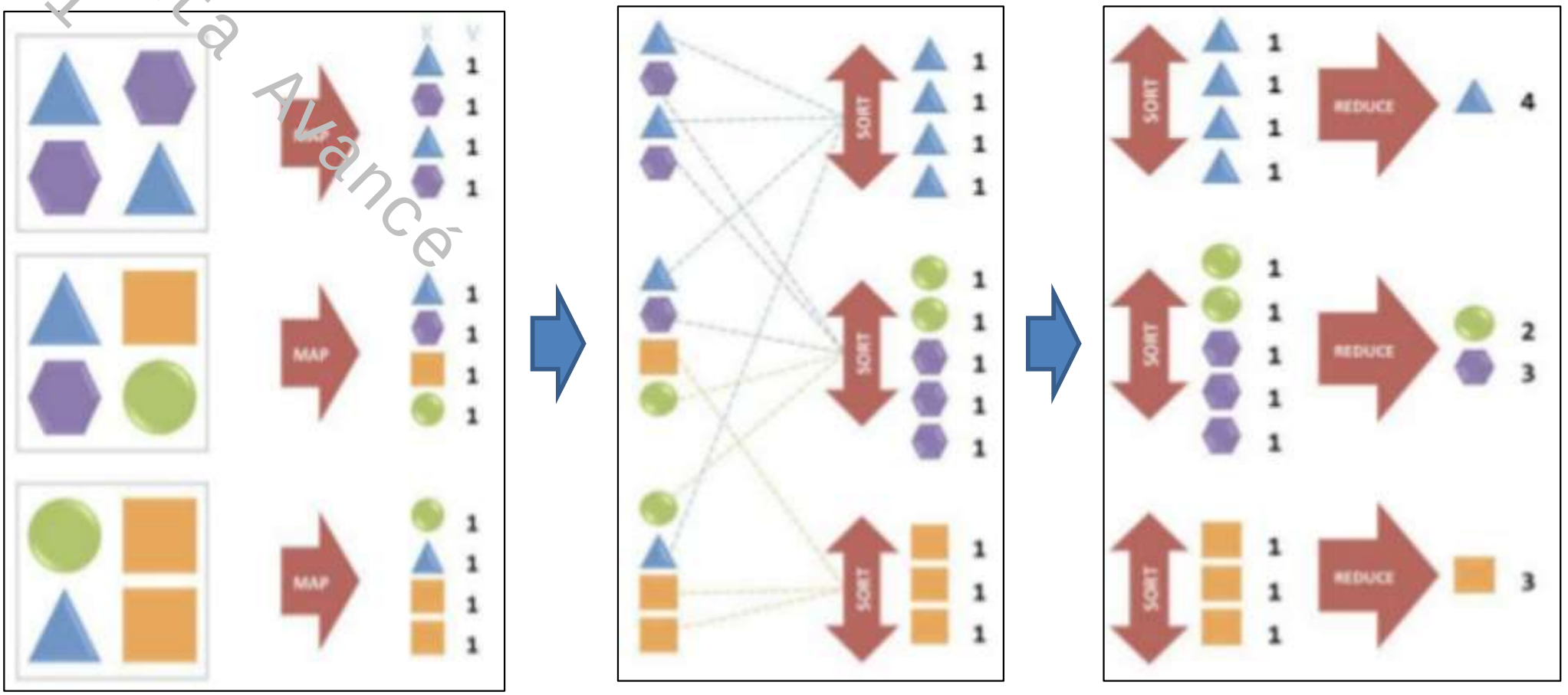
- MapReduce Paradigm :

- Parallel processing
  - ✓ Very fast
  - ✓ Clustering of medium-performance machines
  - ✓ Suitable for real-time applications

MapReduce:
Visual example



MapReduce:
Visual example



MAP **Shuffle & Sort** **Reduce**

- **Principle:**

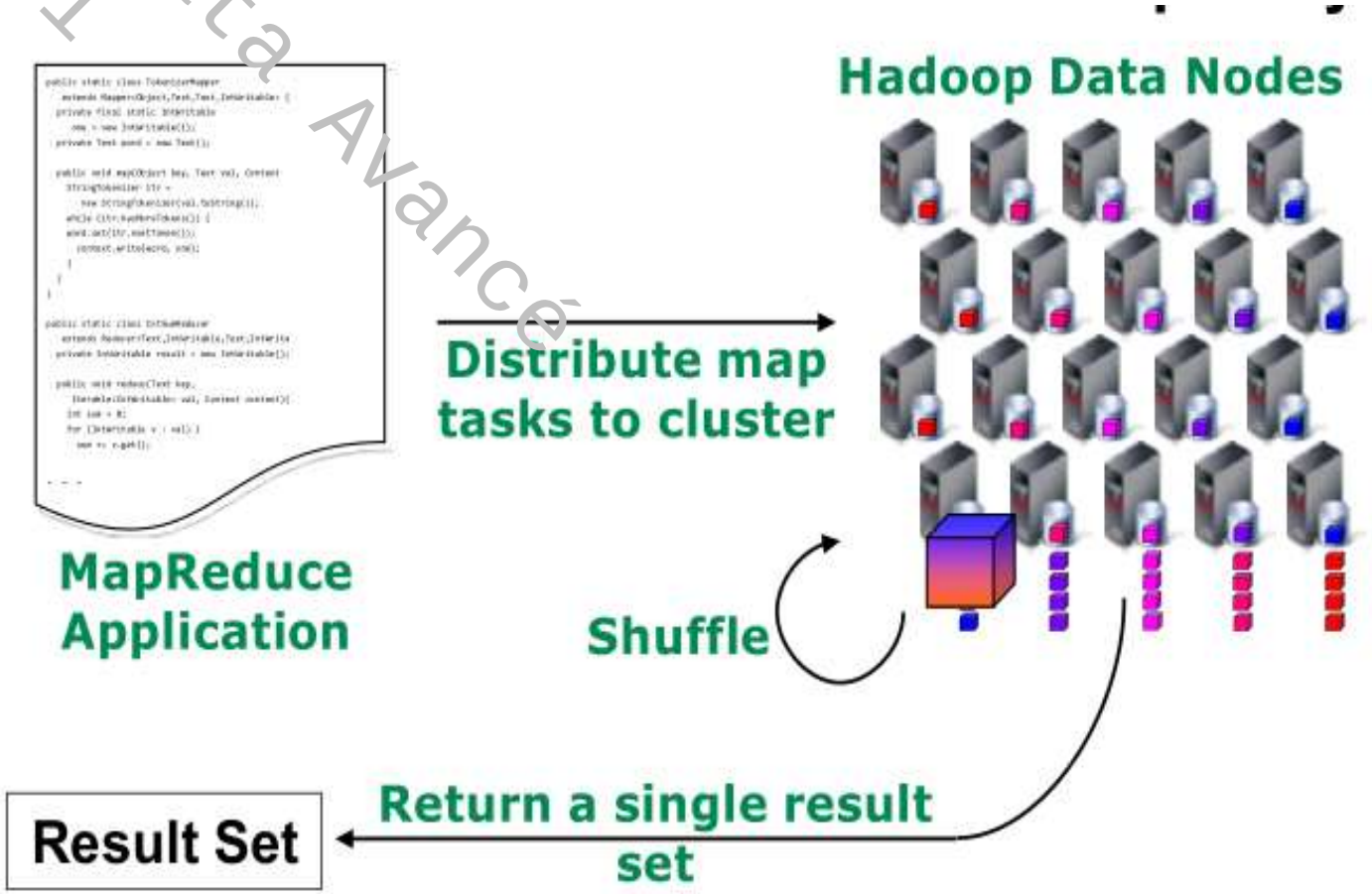
  - Mapreduce is an **algorithmic model**

  - It provides a **"divide & conquer"** functional programming style that automatically divides data processing into tasks and isolates them on the nodes of a cluster.

  - This division is carried out in **3 phases** (or 3 stages):

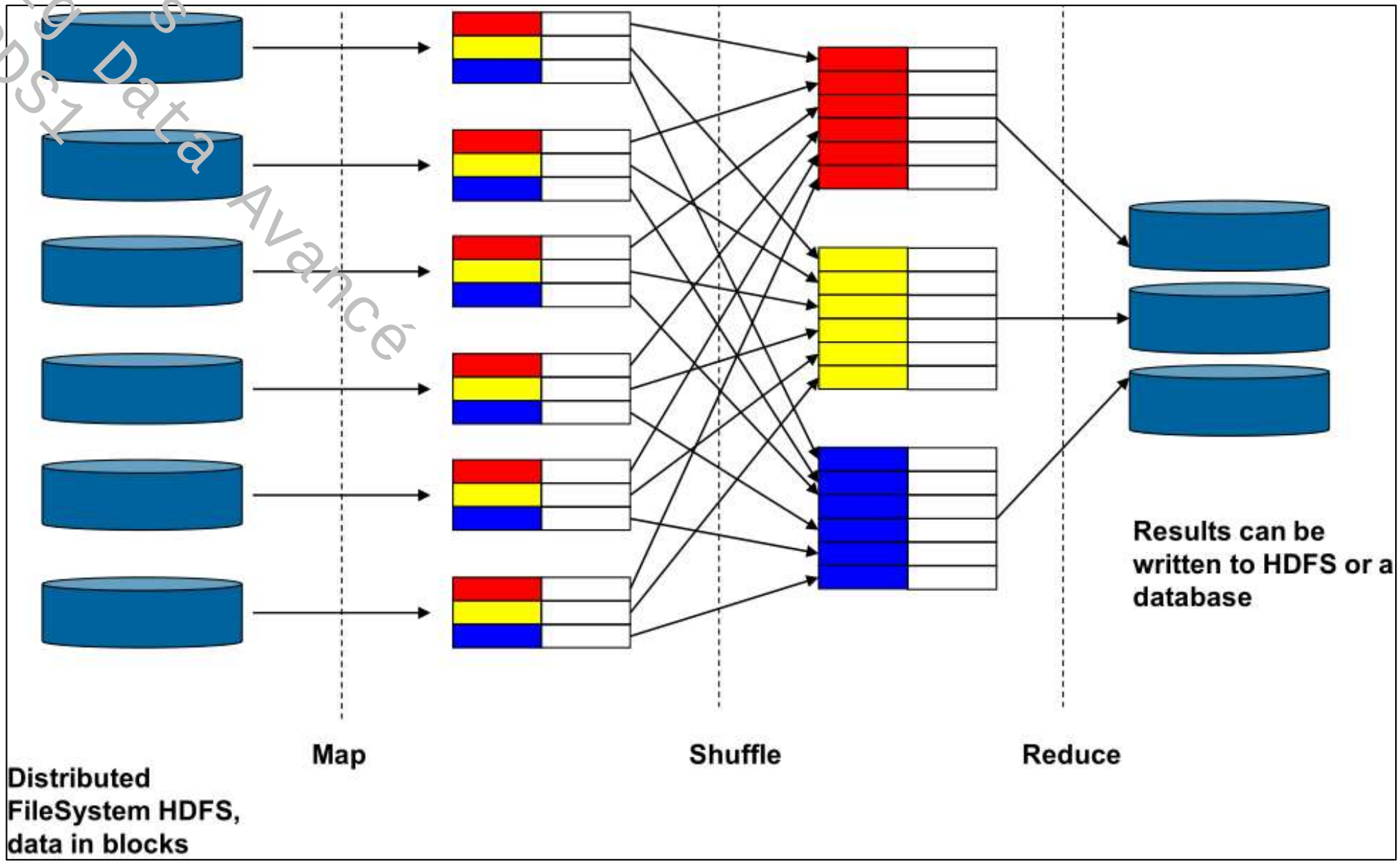
    Map() phase
    Shuffle() phase
    Reduce() phase.

MapReduce:
Principle



1. Map Phase
   (break job into small parts)
2. Shuffle
   (transfer interim output for final processing)
3. Reduce Phase
   (boil all output down to a single result set)

MapReduce:
Principle



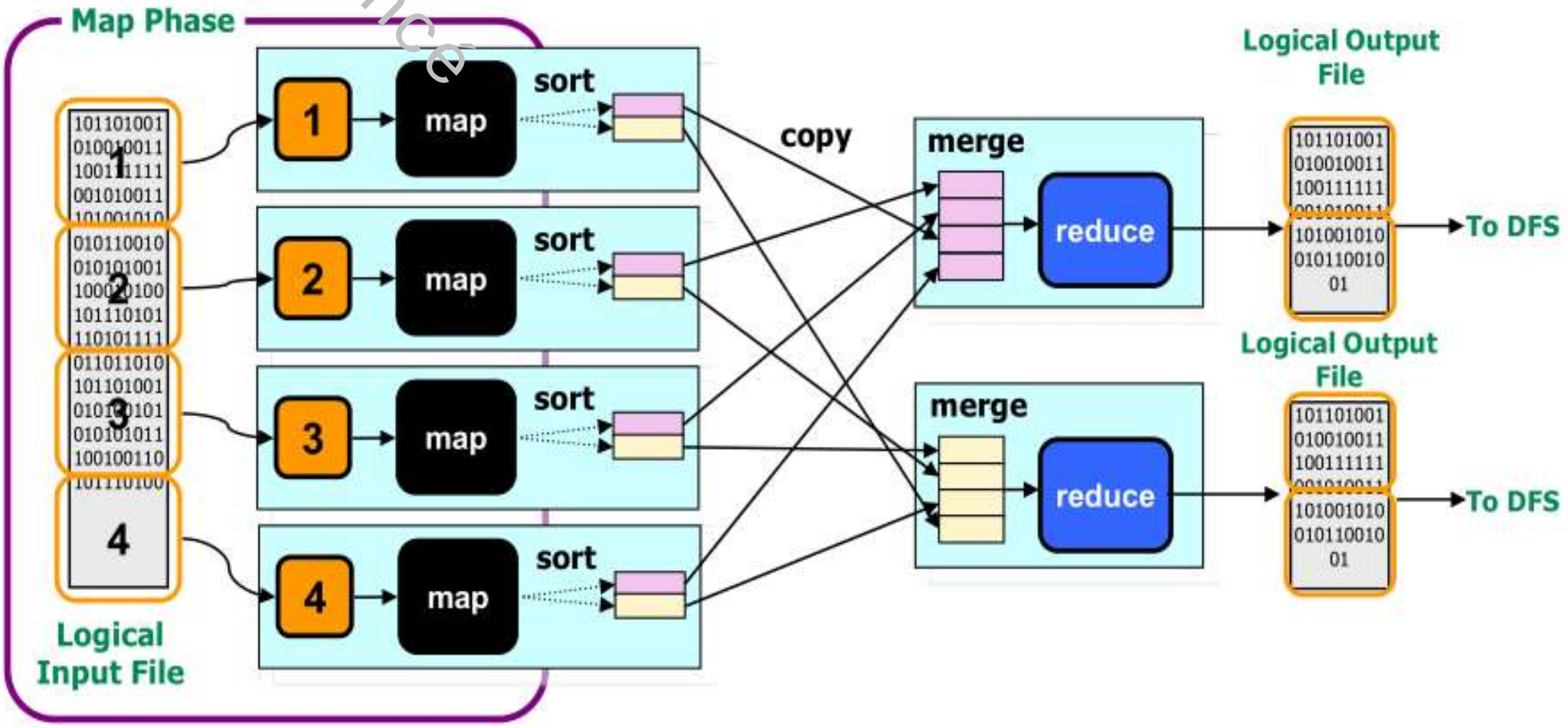
1- File blocks (stored on different DataNodes) in HDFS are read and processed by the Mappers.

2- The output of the Mapper processes are shuffled (sent) to the Reducers (one output file from each Mapper to each Reducer); the files here are not replicated and are stored local to the Mapper node.

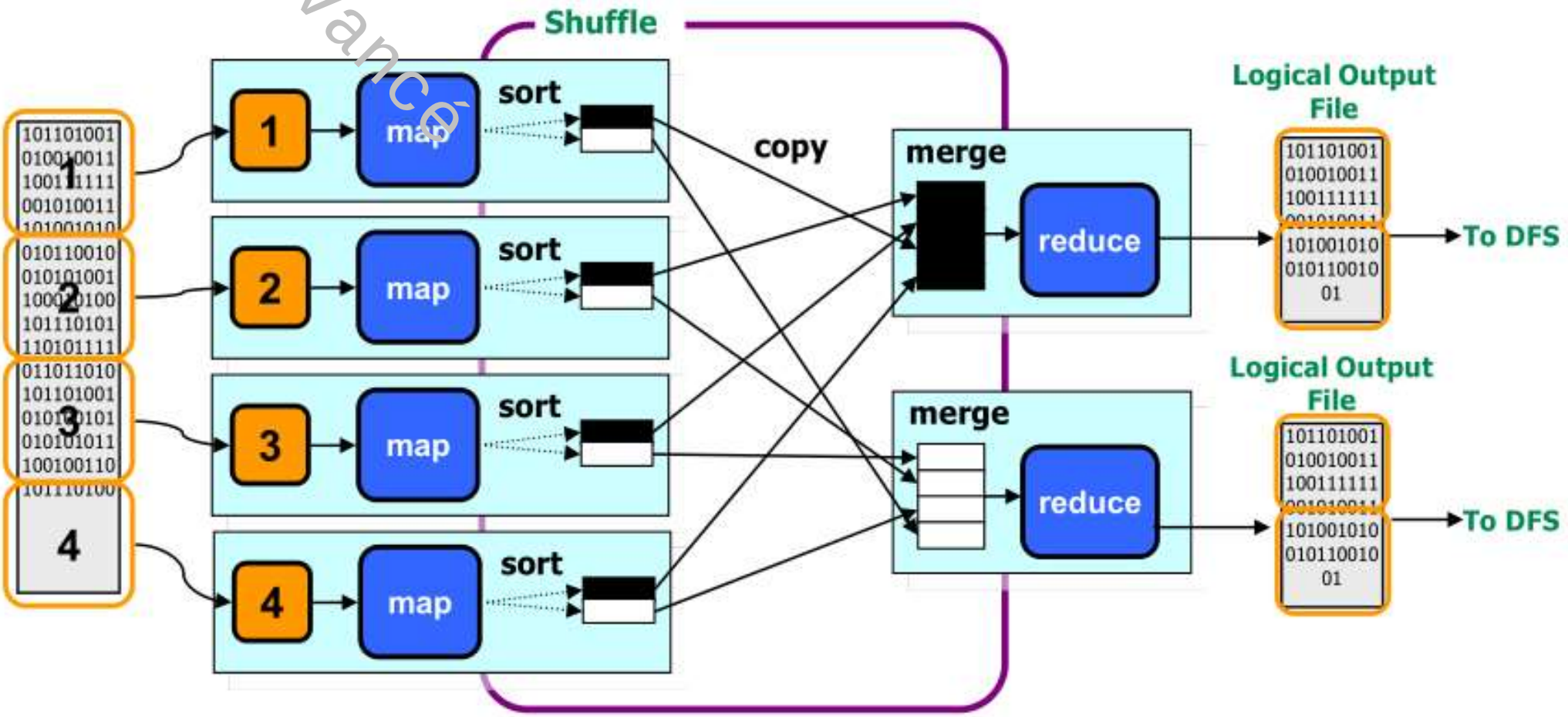
3- The Reducers produces the output and that output is stored in HDFS, with one file for each Reducer.

MapReduce: Principle

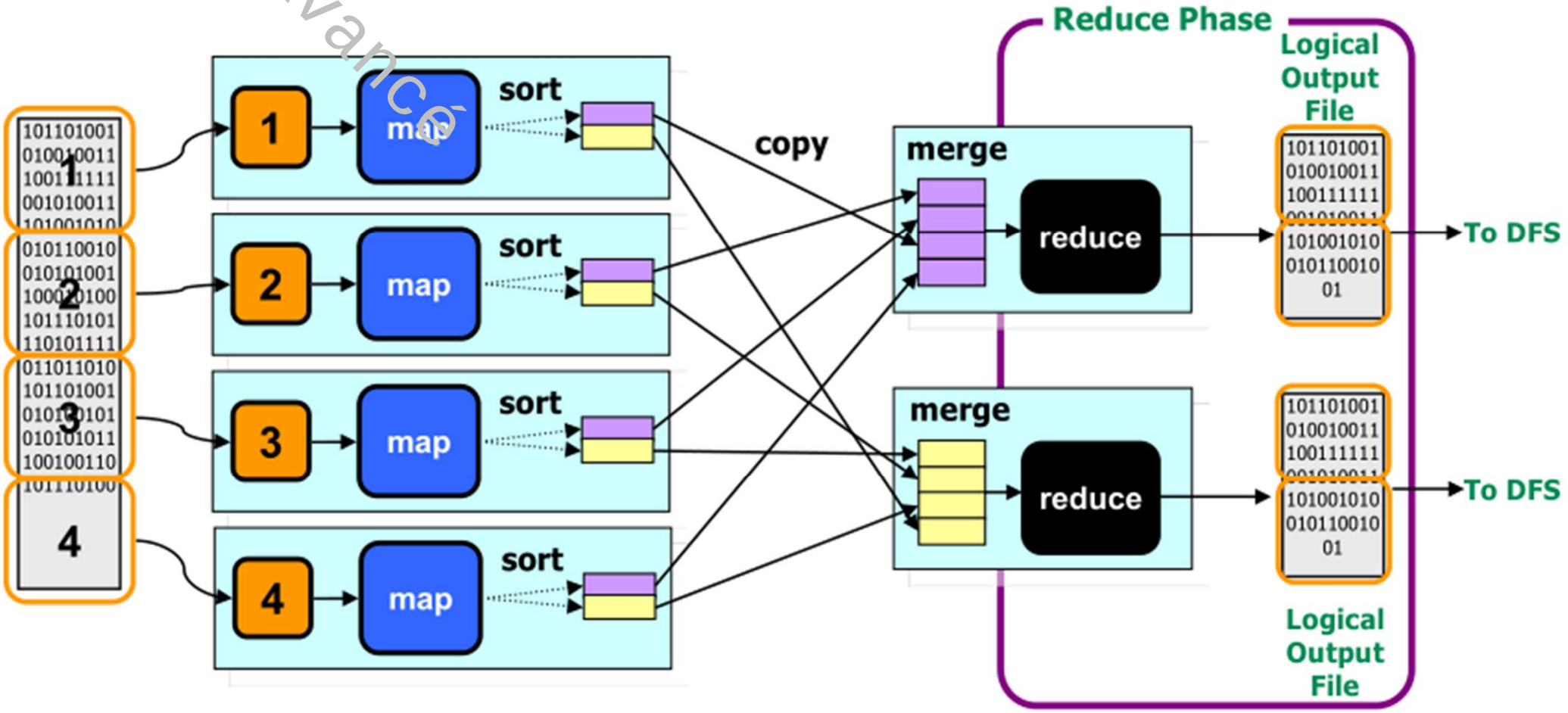
- **Map() phase**
  - Mappers
    - Small program (typically), distributed across the cluster, local to data
    - Handed a portion of the input data (called a split)
    - Each mapper parses, filters, or transforms its input
    - Produces grouped **<key,value>** pairs



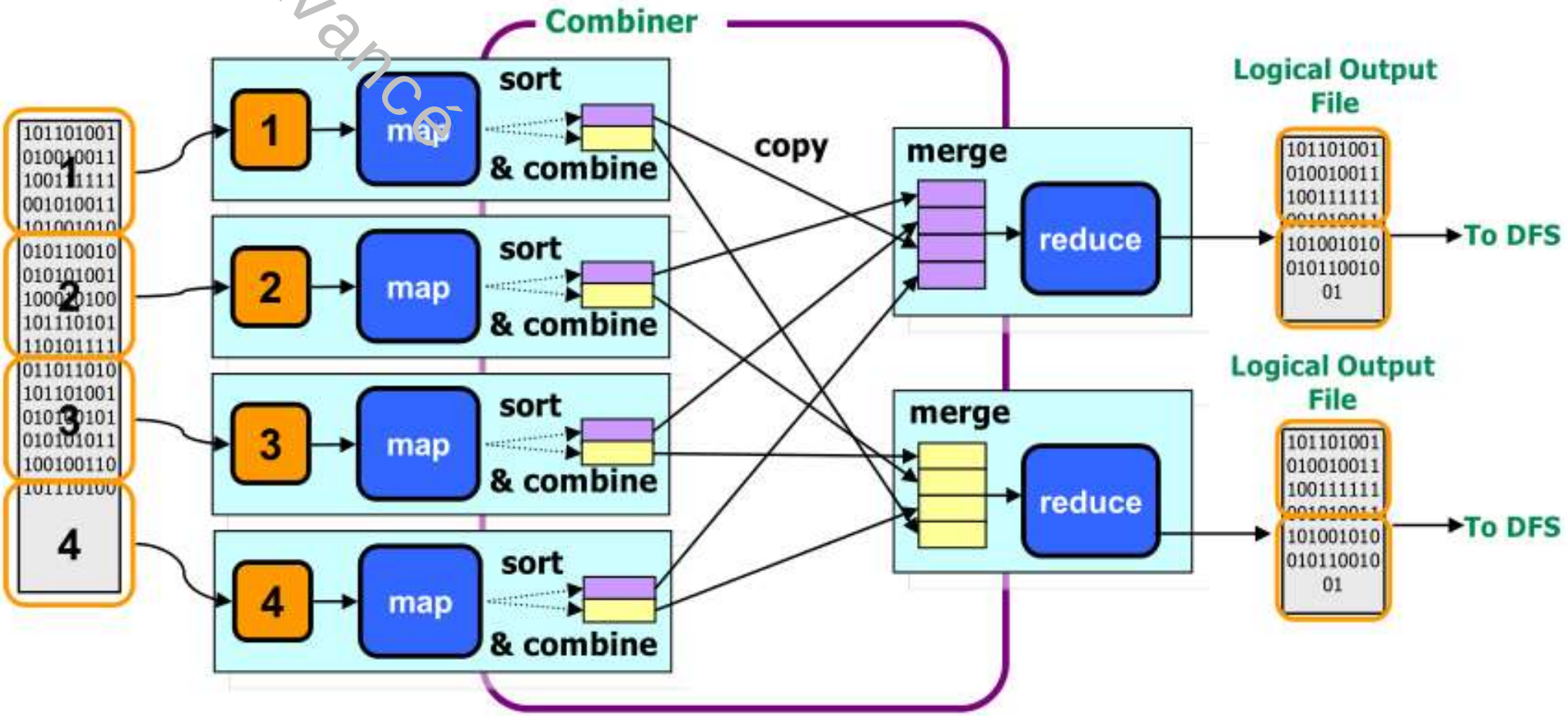
- **Shuffle() phase**
  - The output of each mapper is locally grouped together by **key**
  - One node is chosen to process data for each **unique key**
  - All of this movement (**shuffle**) of data is transparently orchestrated by MapReduce



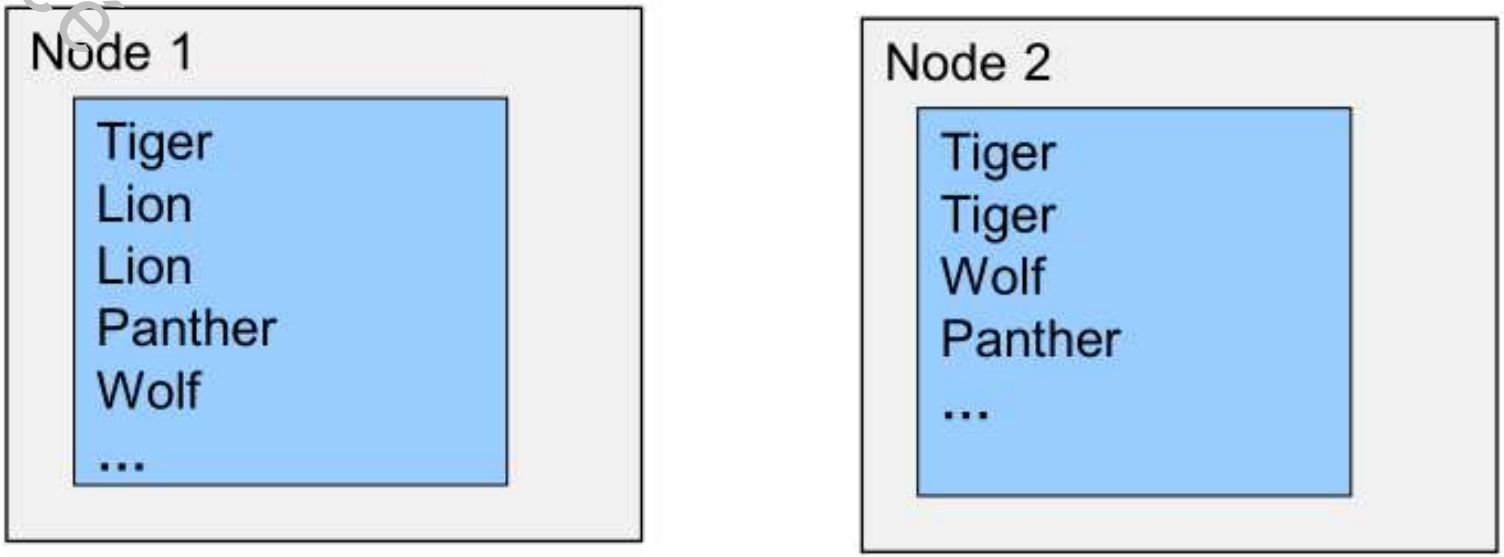
- **Reduce() phase**
  - Reducers
    - Small programs (typically) that **aggregate** all of the values for the key that they are responsible for
    - Each reducer writes output to its own file



- **Combiner() phase (optional)**
  - The data that will go to each reduce node is **sorted and merged** before going to the reduce node, pre-doing some of the work of the receiving reduce node in order to **minimize network traffic** between map and reduce nodes.

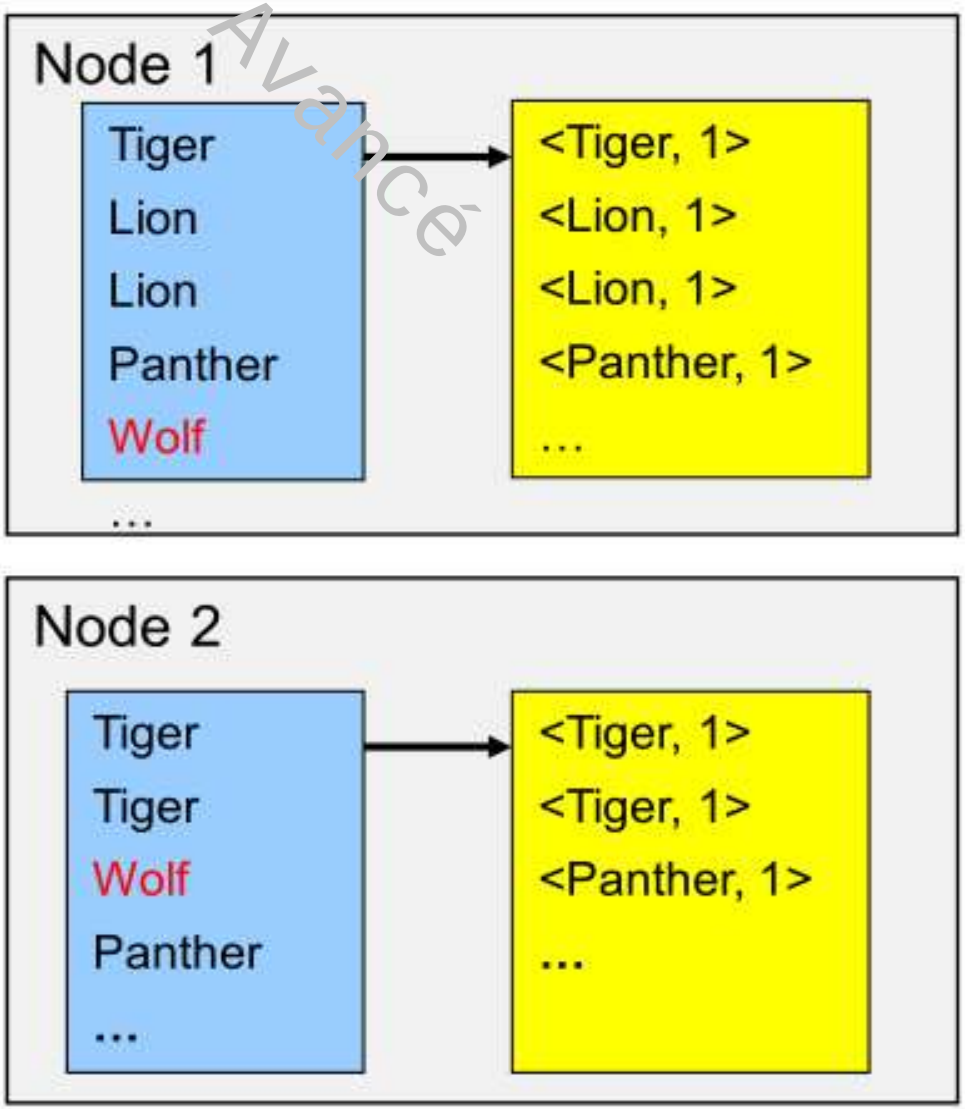


- **Word Count example**
  - Suppose that we want to retrieve as, a list, the total number of big cat animal from one large file
  - For simplicity, we suppose that the file was divided into two blocks (splits)
  - We suppose that animal names are listed. There is one animal name per line in the files.



Since the blocks are held on different nodes, software running on the individual nodes process the blocks separately.

- **Mapper**
  - There are two requirements for the Map task:
    - filter out the non big-cat rows
    - prepare count by transforming to **<Text(name), Integer(1)>**



The Map Tasks are executed locally on each split

- The Map step shown does the following processing:

  - each Map node reads its own "split" (block) of data

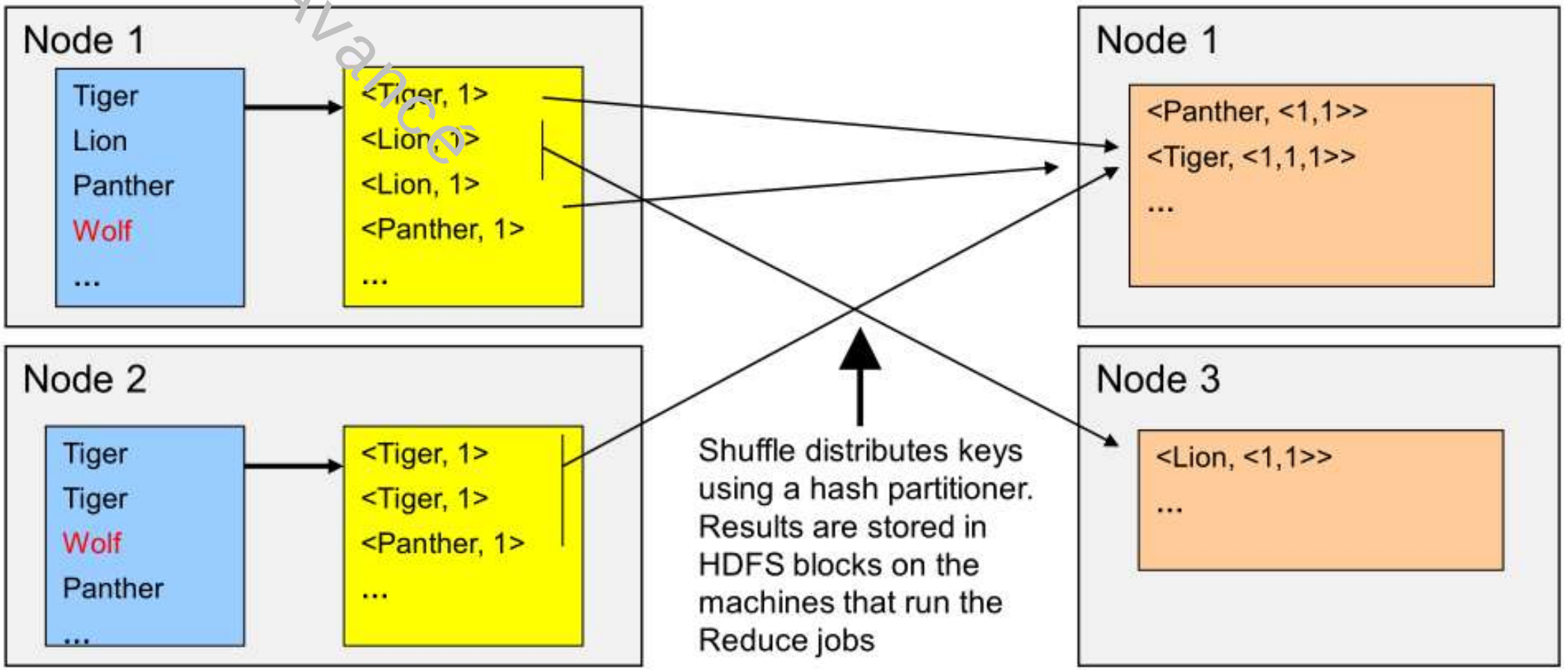
  - the information required (in this case, the names of animals) is extracted from each record (in this case, one line = one record)

  - data is filtered (keeping only the names of cat family animals)

  - key-value pairs are created (in this case, key = animal and value = 1)

  - key-value pairs are accumulated into locally stored files on the individual nodes where the Map task is being executed

- **Shuffle**
  - Shuffle moves all values of one key to the same target node
  - Distributed by a Partitioner Class (normally hash distribution)
  - Reduce Tasks can run on any node - here on Nodes 1 and 3



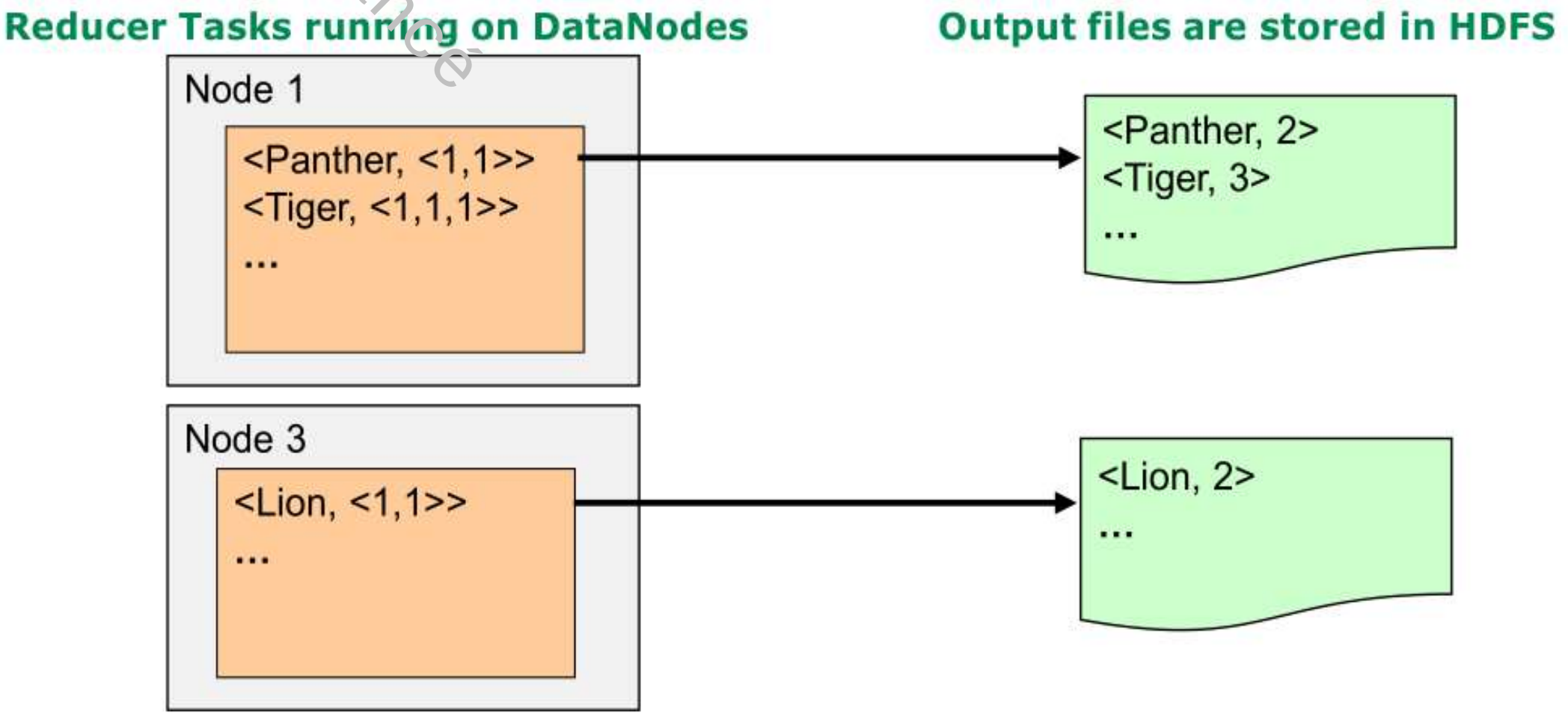
- Shuffle distributes the key-value pairs to the nodes where the Reducer task will run. Each Mapper task produces **one file** for each Reducer task. A hash function running on the Mapper node determines which Reducer task will receive any particular key-value pair. All key-value pairs with a particular key will be sent to the same Reducer task.

- Reduce tasks can run on any node, either different from the set of nodes where the Map task run or on the same DataNodes. In our example, Node 1 is used for one Reduce task, but a new node, Node 3, is used for a second Reduce node.

-There is no relation between the number of Map tasks (generally one node for each block of the file(s) begin read) and the number of Reduce tasks. Commonly the number of Reduce tasks is smaller than the number of Map tasks.

MapReduce:
Word count

- **Reducer**
  - The reduce task computes **aggregated** values for each key
  - Normally the output is written to the HDFS
  - Default is one output part-file per Reduce task
  - Reduce tasks aggregate all values of a specific key (in our case, the count of the particular animal type)



- The Reduce step does the following processing:

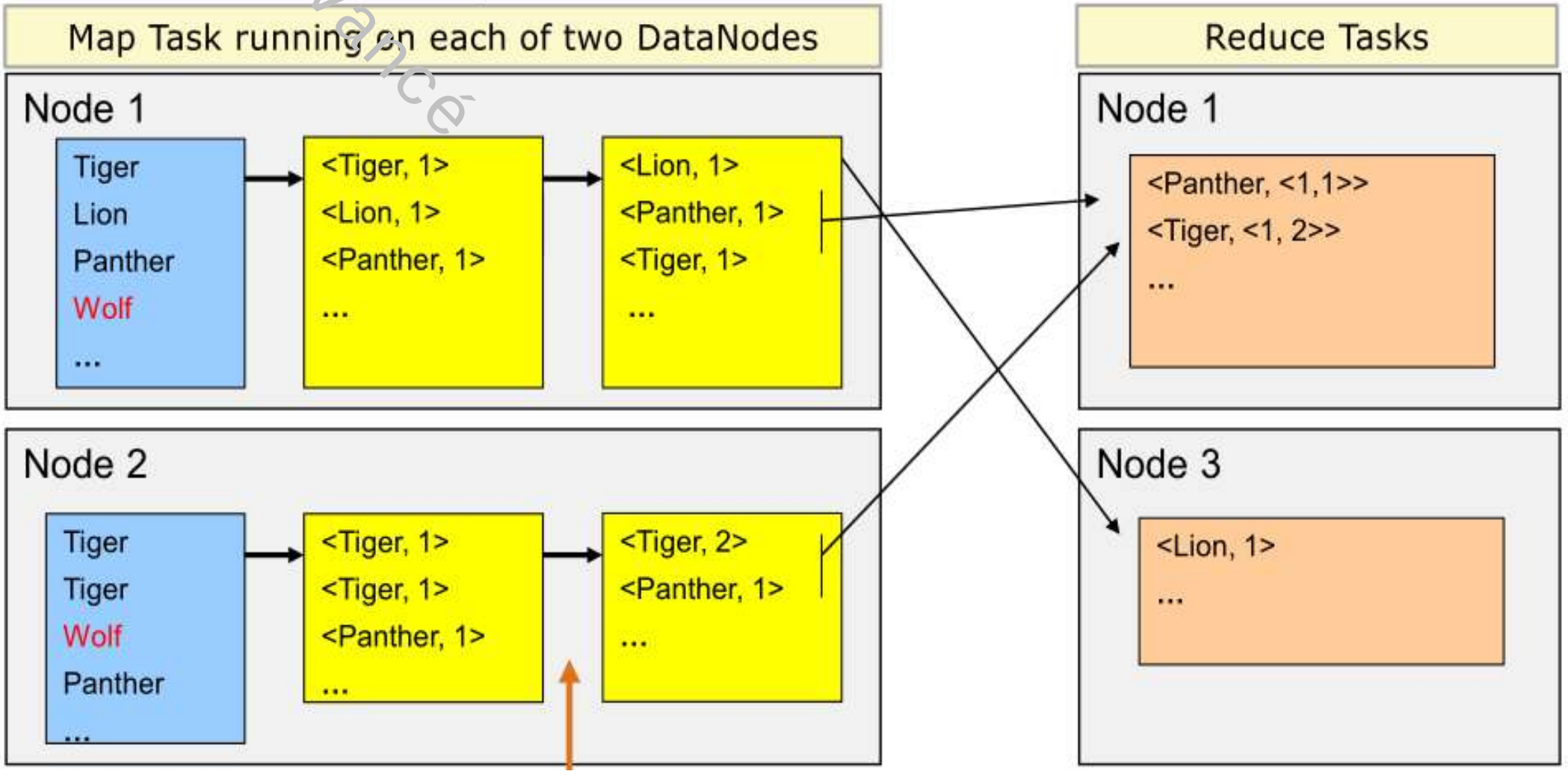
  - Each Reduce node process the data sent to it from the various Map nodes

  - This data has been previously sorted (and possibly partially merged)

  - The Reduce node aggregates the date; in the case of WordCount, it sums the counts received for each particular word (each animal in this case)

  - One file is produced for each Reduce task and that is written to HDFS where the blocks will automatically be replicated

- **Combiner (optional)**
  - For performance, an aggregate in the Map task can be helpful
  - Reduces the amount of data sent over the network
  - Also reduces Merge effort, since data is premerged in Map
  - Done in the Map task, before Shuffle



- **Combiner (optional)**

  - The Combiner phase is optional. When it is used, it runs on the Mapper node and preprocesses the data that will be sent to Reduce tasks by pre-merging and pre- aggregating the data in the files that will be transmitted to the Reduce tasks.

  - The Combiner thus reduces the amount of data that will be sent to the Reducer tasks, and that speeds up the processing as smaller files need to be transmitted to the Reducer task nodes.

- Writing a Map Reduce program comes down to:

  - 1 Choosing a **way** of slicing the data so that **Map** and **Reduce** operations are parallelizable

  - 2 Choose the **key** to use for the problem

  - 3 Write the program for the Map operation

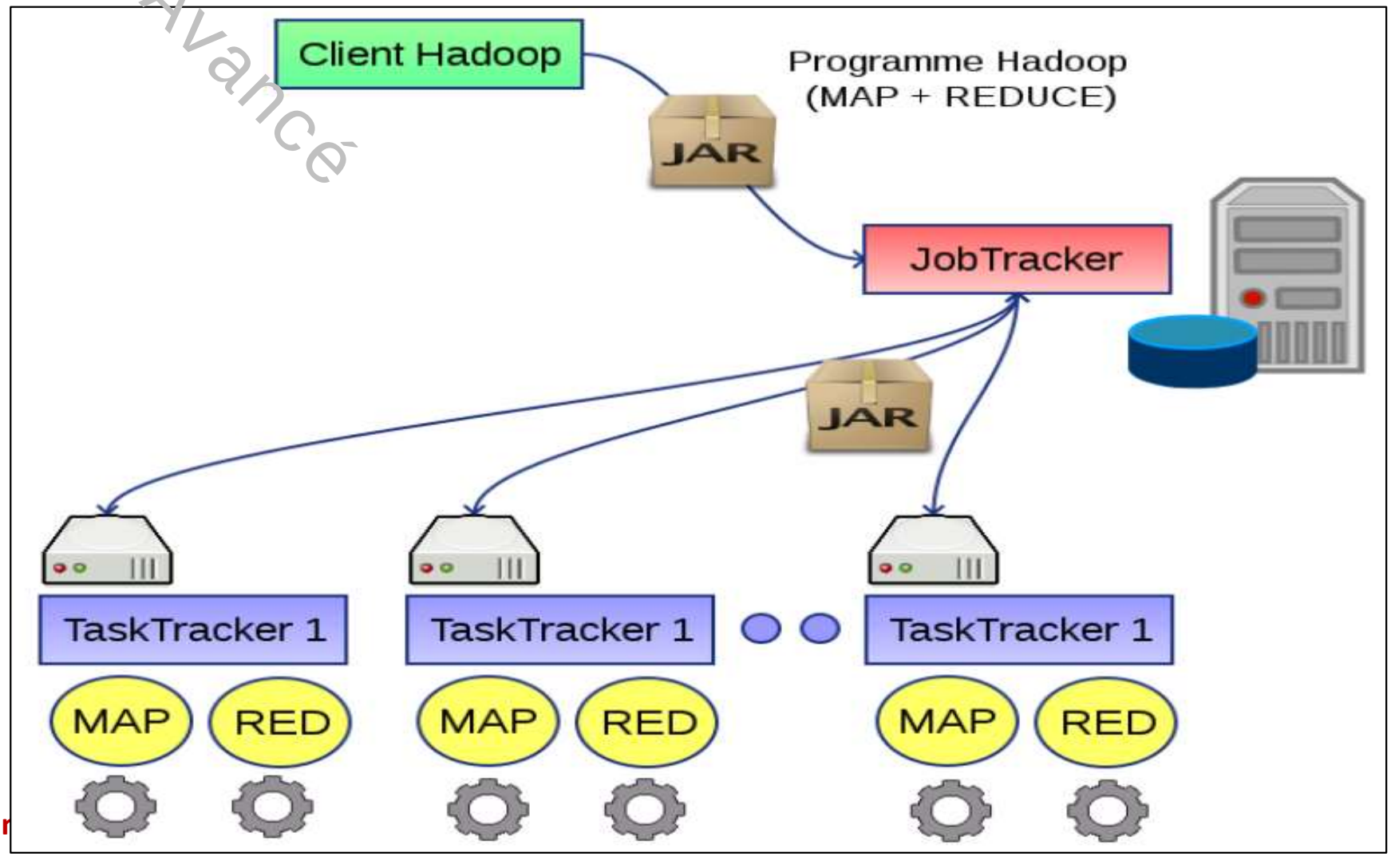
  - 4 Write the program for the Reduce operation

- Architecture: Based on **two servers**:

  - The **JobTracker** (unique on the cluster).
  Receives the map/reduce tasks to be executed (in the form of a **Java.jar** archive) and organizes their execution on the cluster.

  - The **TaskTracker** (several per cluster).
  Executes the map/reduce job (in the form of map and reduce tasks with associated input data).
  Each TaskTracker is a computing unit in the cluster.

- The **JobTracker server** communicates with HDFS; it knows where the input data for the map/reduce program is and where the output data should be stored. It can therefore optimize the distribution of jobs according to the associated data.



- JobTracker monitors the status of the TaskTrackers by means of heartbeat packets sent continuously by the TaskTrackers.
- In the event of a TaskTracker failure (missing heartbeat or failed job), the JobTracker redistributes the job to another node.
- During the execution of a task, it is possible to obtain detailed statistics on its progress (current stage, progress, estimated time to completion, etc.) via the hadoop console client.
- Each node has a certain number of predefined **slots** (Map Slots and Reduce Slots).

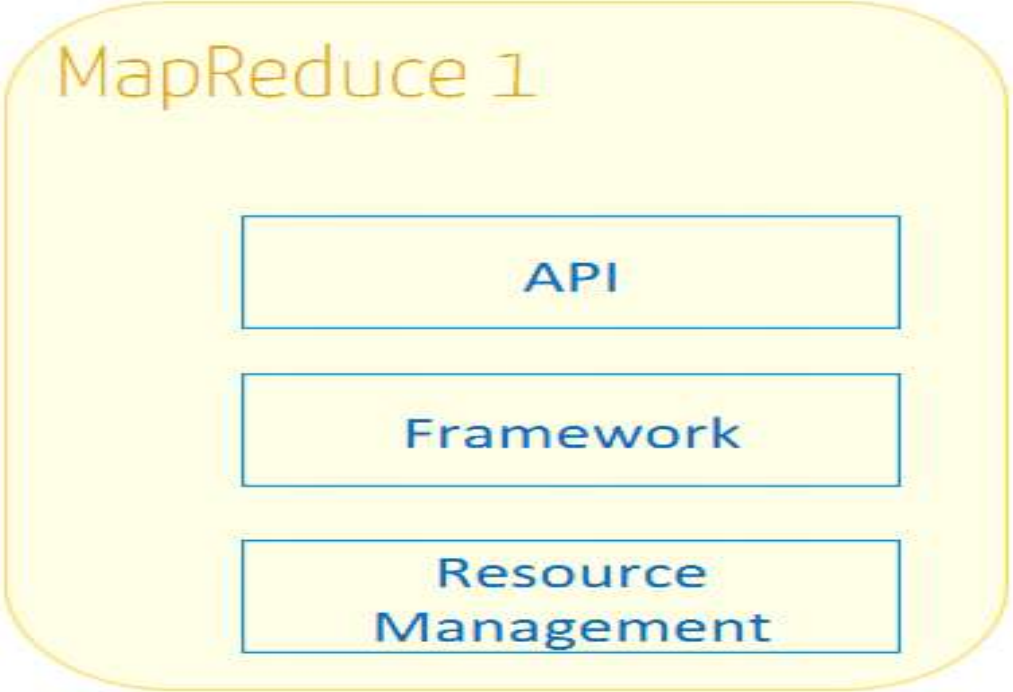
  A slot is a unit of execution that represents the task tracker's capacity to execute a task (map or reduce) individually, at a given time.
- **Note:** The two "unique" NameNode and JobTracker servers are often active on the same machine: the cluster master node.

- MapReduce v1 components

- API: enables programmers to write MapReduce applications

- Framework: for executing Map, Shuffle/Sort and reduce jobs

- Resource Management Infrastructure: manages cluster nodes, allocates resources and ensures job scheduling



- The **Job Tracker server** runs on the master machine. It also performs several tasks (scheduling, resource management, etc.).

  - Scalability problem: Job Tracker performance limits the number of DataNodes supervised (number of nodes per cluster limited to 4000)

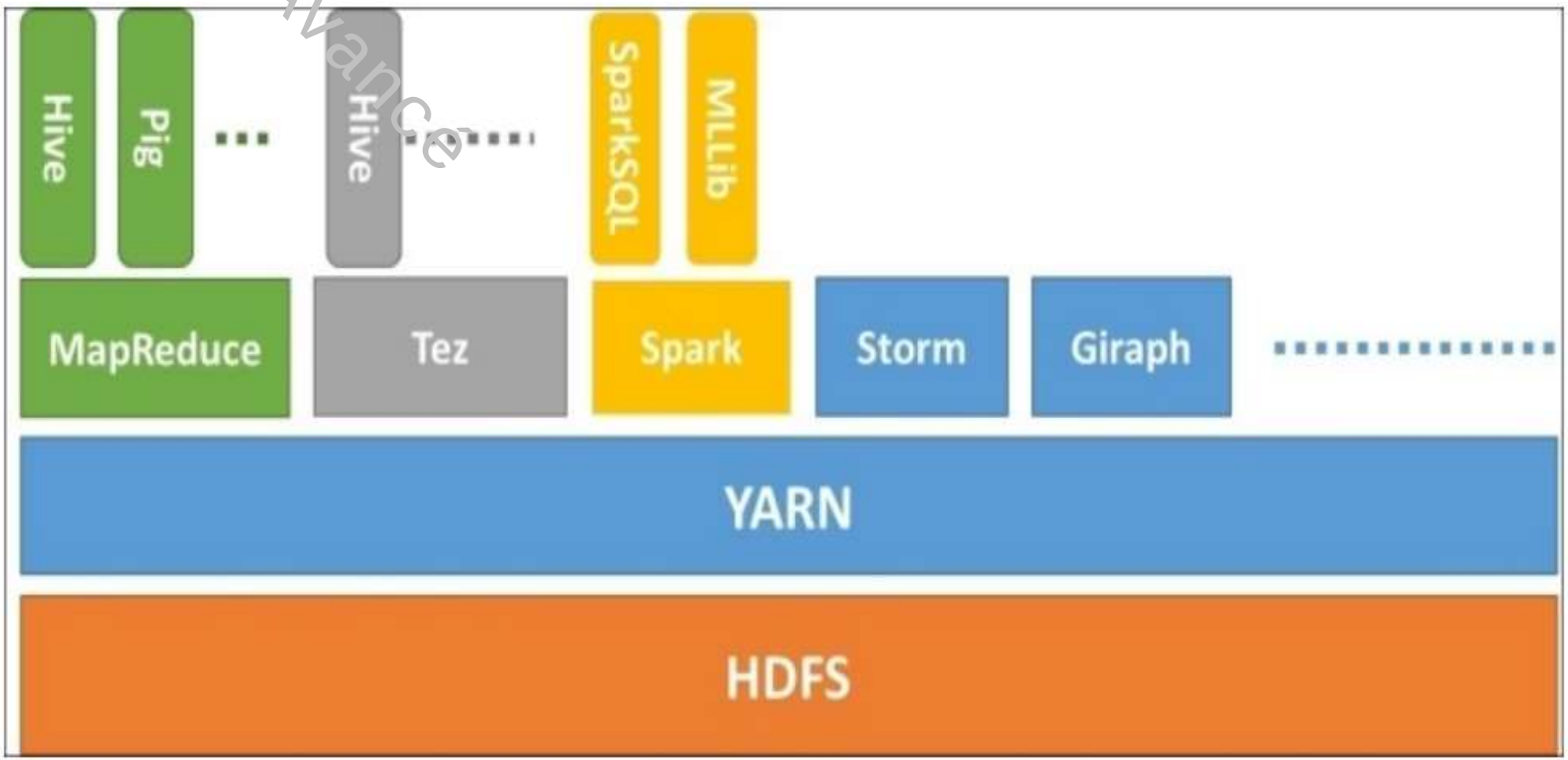
  - Job Tracker crashes ➔ reinitialization of all Task Trackers (availability problem)

  - The number of slots is predefined ➔ Problem exploiting node capacity

  - **Job Tracker is highly integrated with Map Reduce**

  ➔ Interoperability problem (impossible to run non-MapReduce applications on HDFS)

- Purpose of YARN
  - Allows users to use several distributed applications that coexist side by side with MapReduce applications and **share** the same cluster and HDFS file system.



- In Mapreduce v1, there are two types of daemon which control the task execution process: jobtracker and one or more tasktrackers.

  - **Jobtracker coordinates all the tasks** executed on the system by ordering the tasks to be executed on the tasktrackers.
  - **The tasktrackers execute** the tasks and send progress reports to the jobtracker, which keeps a record of the overall progress of each task. If a task fails, the jobtracker can reschedule it on another tasktracker.

- In Mapreduce v1, **the jobtracker handles**
  - scheduling tasks (matching tasks with Tasktracker) and
  - monitoring task progress (task tracking, failed or slow restart).

- **Wit YARN** and Unlike MRv1, the tasks performed by Jobtracker are split into two parts:
  - the **resource manager**
  - **application master** (one for each Mapreduce job).

- Jobtracker is also responsible for storing the job history for completed jobs,

- **Note**: it is possible to run a job history server as a separate daemon to offload the Jobtracker. In YARN, the equivalent role is performed by a **timeline server** (which stores application history).

MRv1 & YARN components

MapReduce 1	YARN
Jobtracker	Resource manager, application master, timeline server
Tasktracker	Node manager
Slot	Container

- **Scalability**
  - In MRv1, a single Jobtracker quickly reaches its tasktracker management limits (4,000 nodes and 40,000 tasks). This is because the Jobtracker has to manage both **tasks** and **resources**.
  - YARN overcomes these limitations with its resource management architecture and split application management (up to 10,000 nodes and 100,000 tasks).
  - Unlike MRv1's Jobtracker, **each instance** of an application (here, a Mapreduce task) **has a dedicated application master**, which runs for the duration of the application.

- **Availability**

- High availability is usually achieved by replicating (duplicating) the daemon state in order to respond to the requested service or in the event of a deadline.

- However, the large amount of complex and rapidly changing state in the Jobtracker memory (each job state is updated every few seconds) makes it very difficult to update Jobtracker services.

- With Jobtracker responsibilities **split** between the **resource manager** and **application master**, high availability is achieved

- **Use**

  - In Mapreduce v1, each Tasktracker is configured with a static allocation of fixed "slot" size, which are divided into Map slots and Reduce slots at the time of configuration.
  - A Map (res. Reduce) **slot can only be used to execute** a Map (res. Reduce) task.
  - In YARN, Resource manager manages a **pool of resources** rather than a **fixed number of slots**.
  - Mapreduce running on YARN will not have the constraint where a Reduce job has to wait because only Map slots are running (a frequent case in MRv1).
  - With MRv2, if the resources of a node to execute a task are available, then it will be executed.
  - YARN resources are not allocated statically (slots). An application can request resources according to its needs.
  - Static resource allocation in MRv1 (indivisible slots):
    too many → resources are wasted
    too few → resources are available, the task in question may fail.

- **Multilocal**

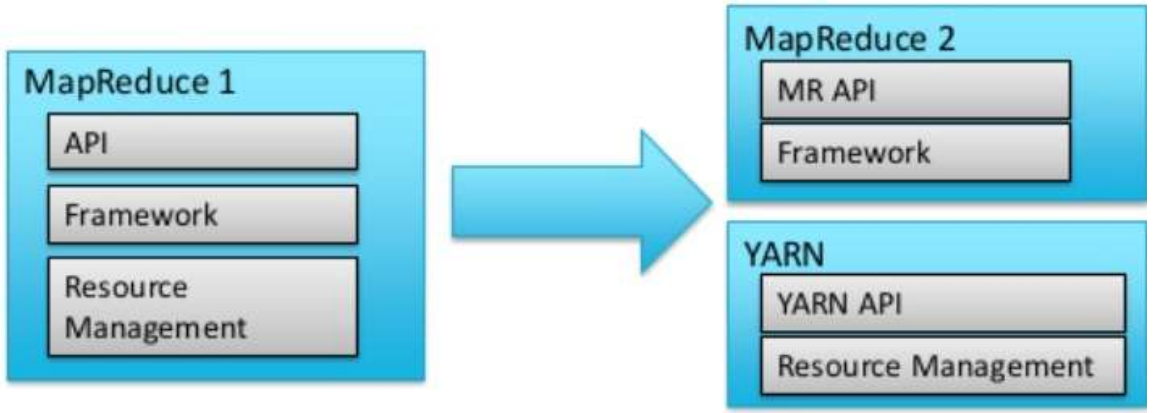
- The biggest advantage of YARN is that it **opens up Hadoop** to other types of distributed application beyond Mapreduce.

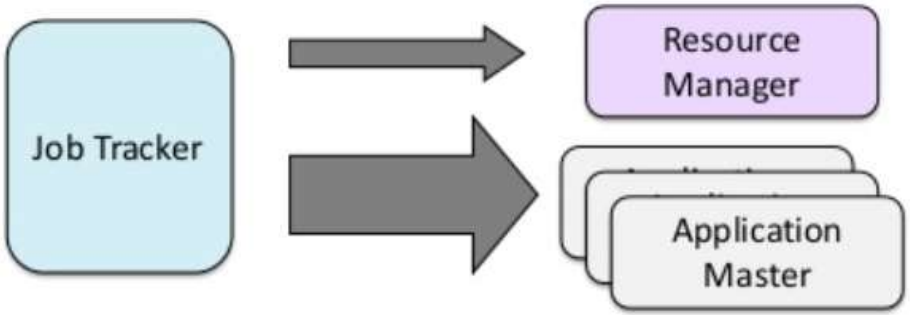
- Mapreduce is just one of many YARN applications.

- It is even possible for users to run different versions of Mapreduce on the same YARN cluster, which makes the process of upgrading Mapreduce more manageable.

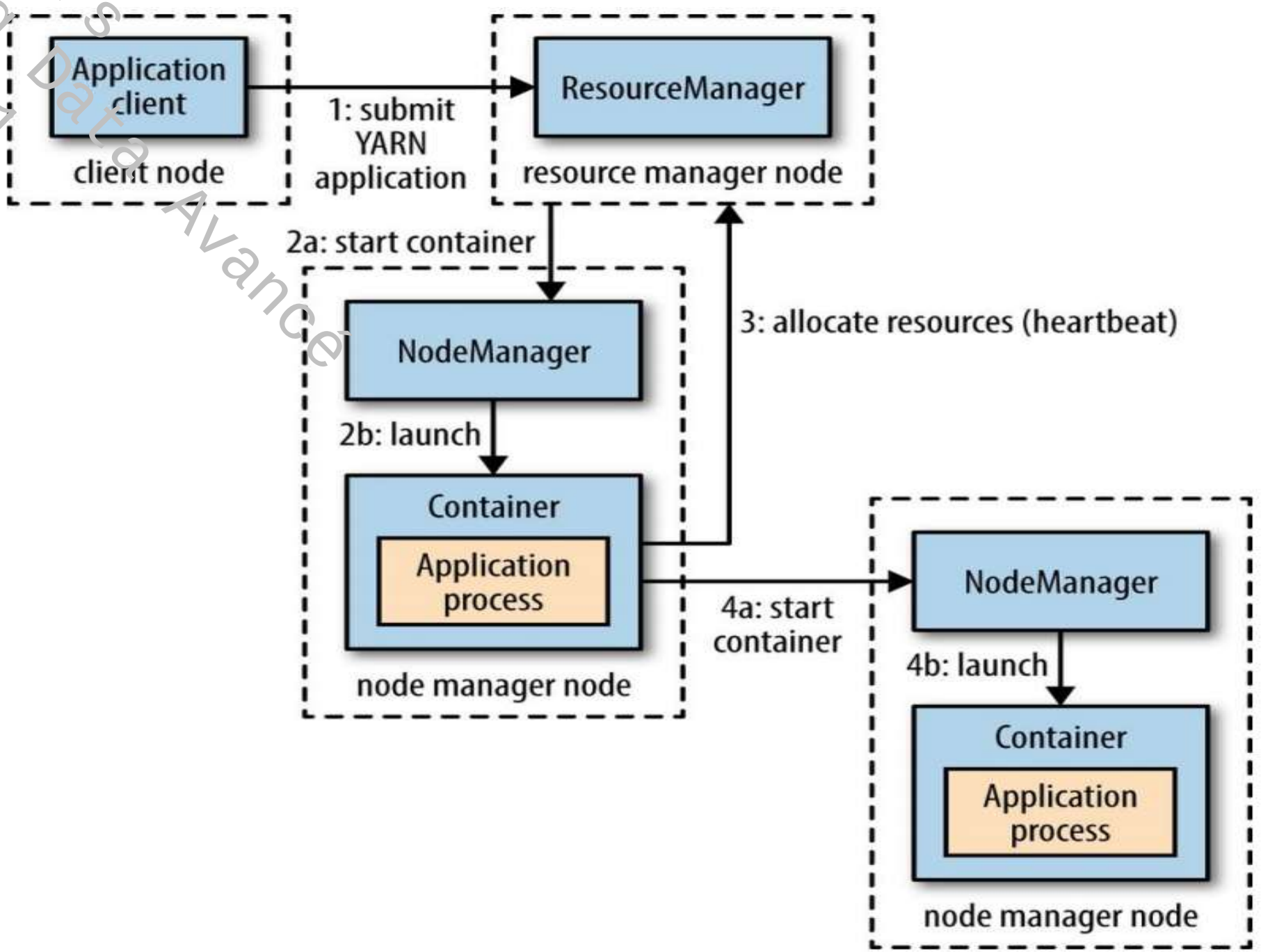
YARN:
Architecture

- MRv2 **assigns** the resource management load to YARN (Yet Another Resource Negotiator)
- MRv2 **separates** resource management from MR task management
- **Supports** both MR and non-MR applications
- Definition of **new demons** (Resource Manager, Node Manager, Application Masters)





YARN: Architecture

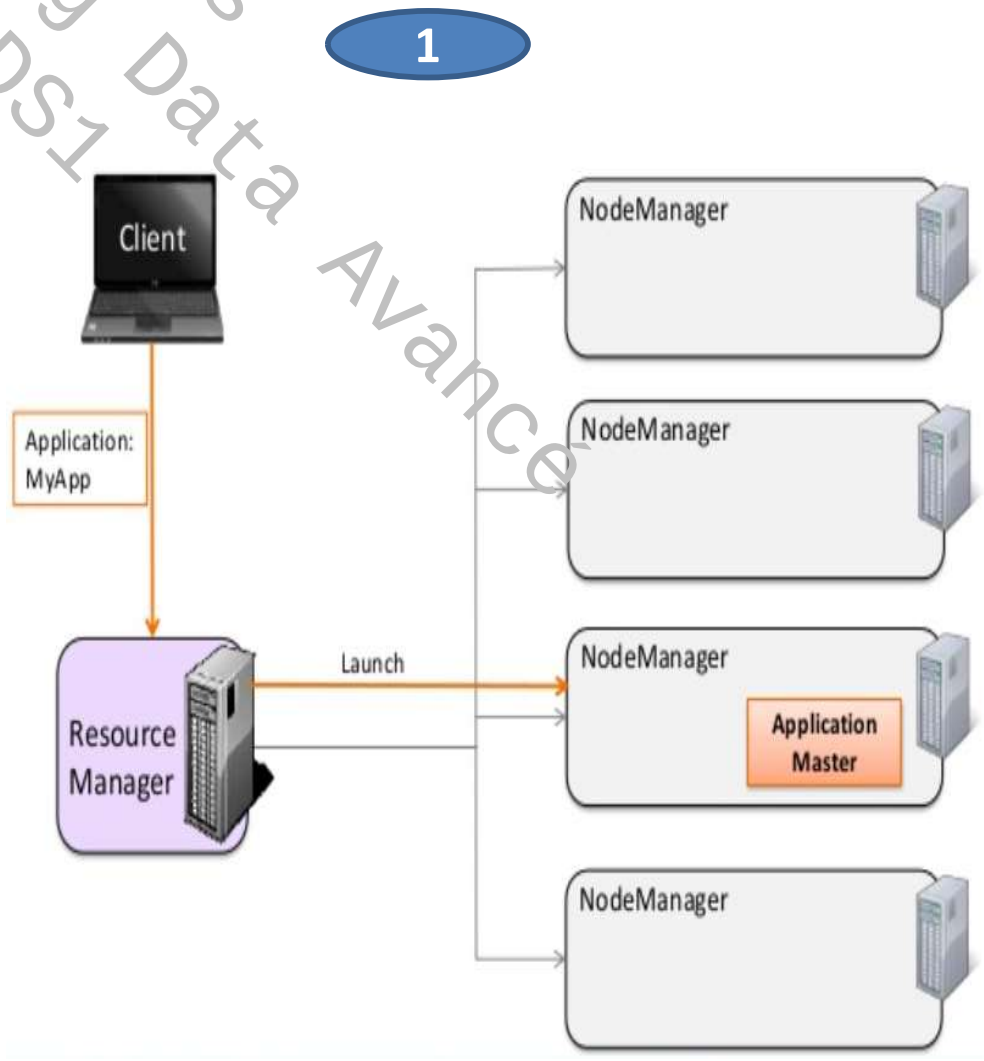


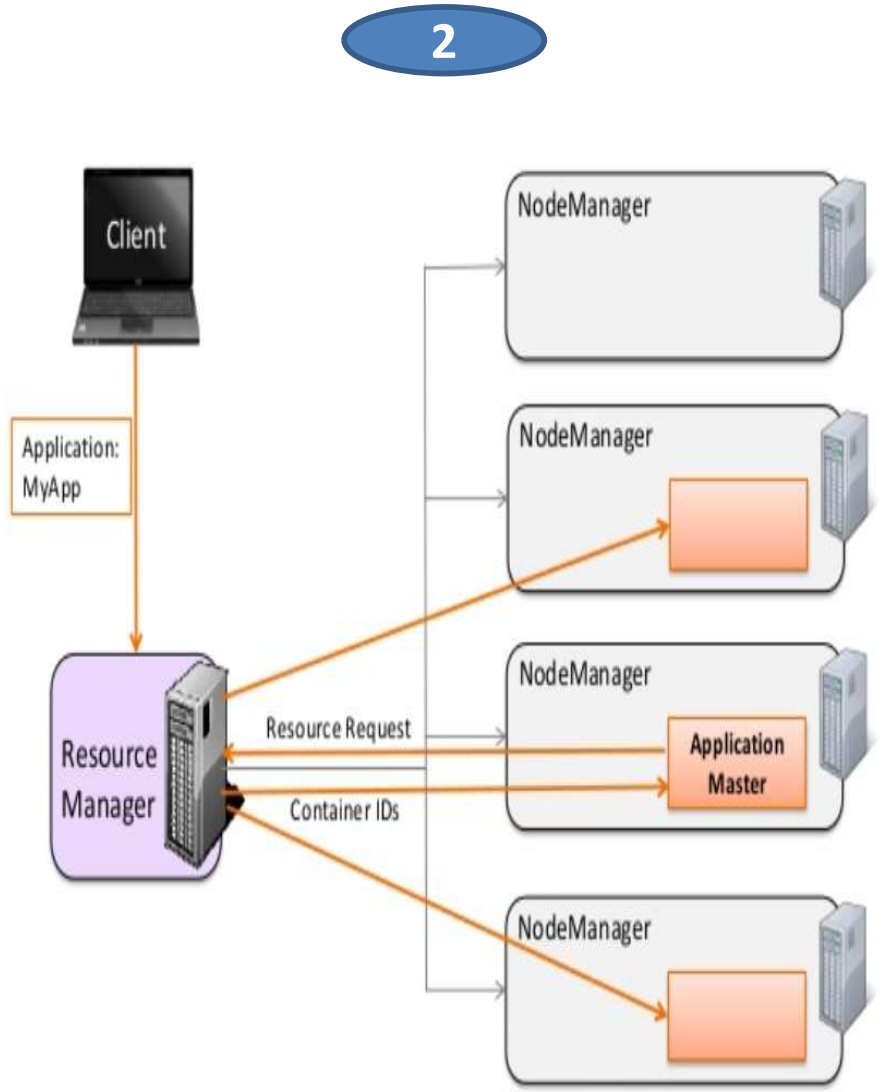
- To launch an application on YARN, a client contacts the **Resource Manager** and asks it to run an **Application Master** main process (step 1).

- Resource Manager then finds a **Node Manager** that can **launch Application Master** in a **container** (steps 2a and 2b).

- What **Application Master** does once it's running depends on the application.
  - It could simply perform a calculation in the container in which it is running and return the result to the client.
  - Or it could request more containers from the resource managers (step 3) and use this to run a distributed calculation (steps 4a and 4b).

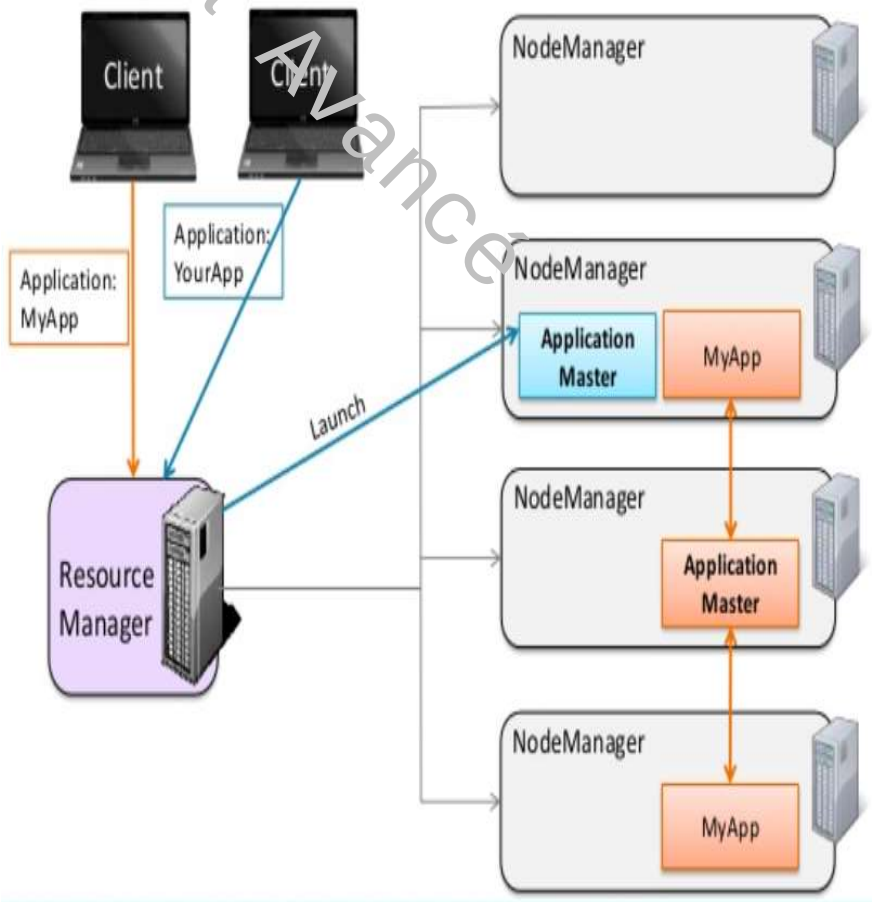
YARN: Architecture



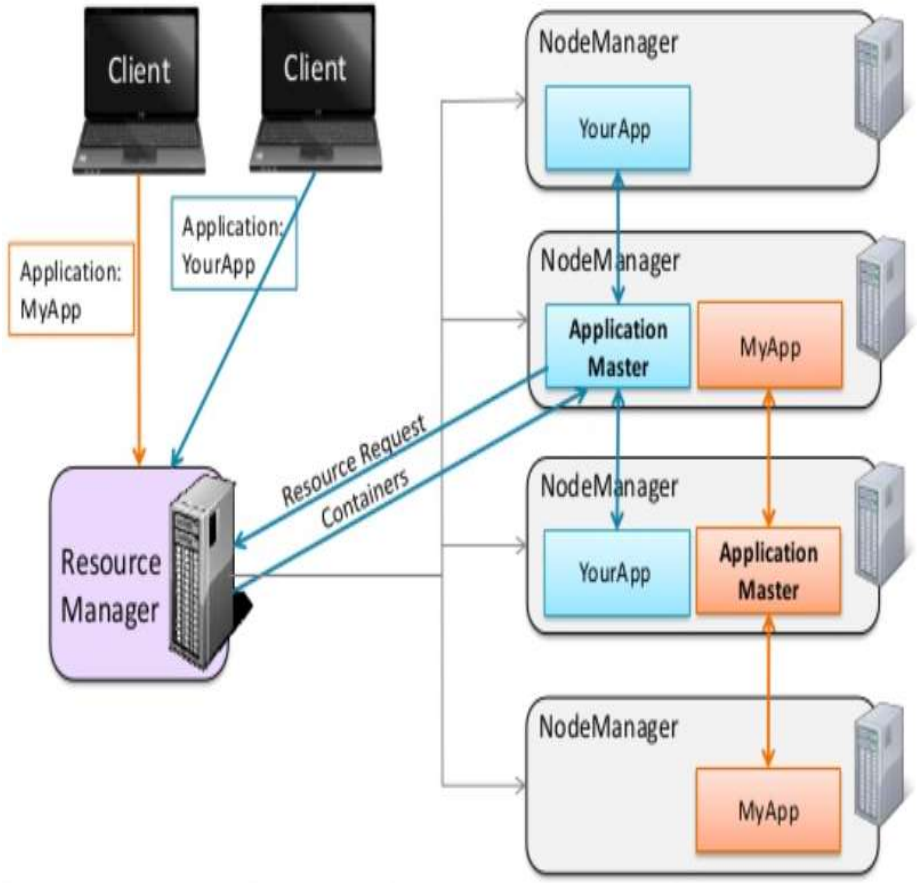


YARN: Architecture

3



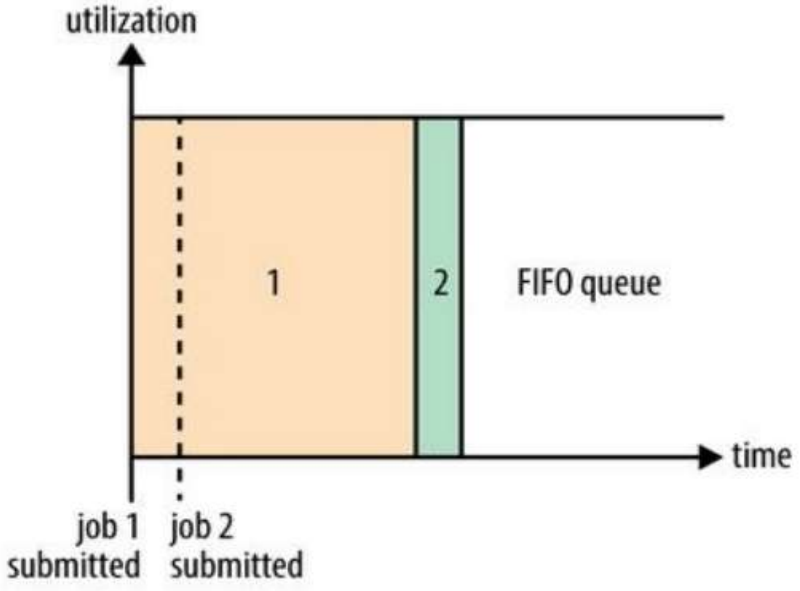
4



YARN:
Resources allocation: FIFO

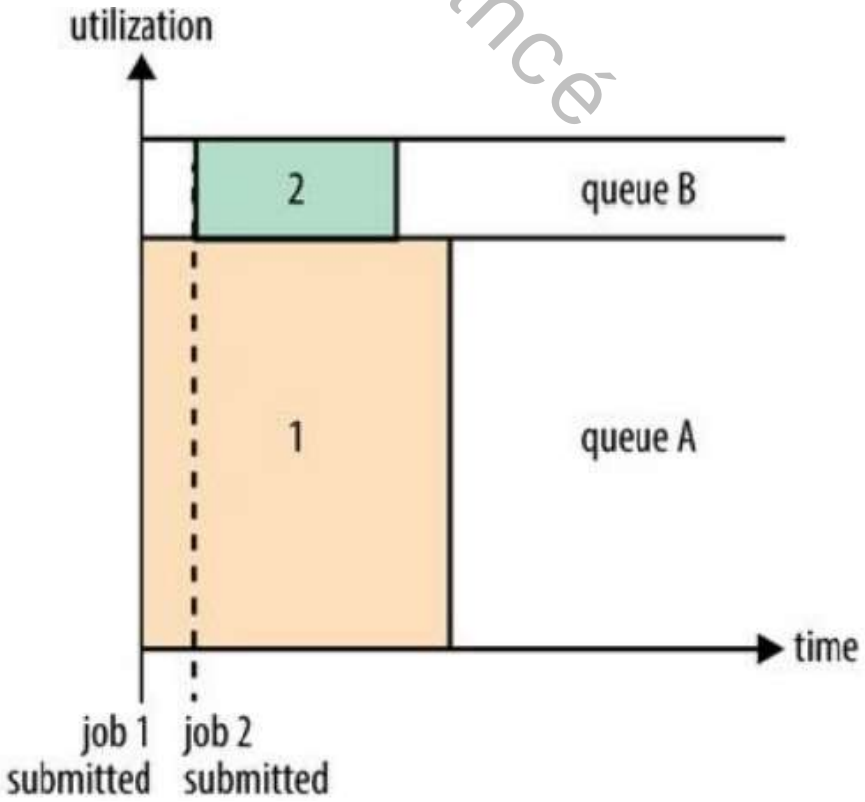
- **Three** resource planners are available in YARN:
  - **FIFO (First Input First Output)**
  - **Capacity**
  - **Fair**

- **FIFO** places requests in a queue and executes them in the order in which they are submitted (first in, first out).
- FIFO is simple and needs no additional configuration, but is not suitable for shared clusters. Large applications will use up all the resources in the cluster, so each application has to wait its turn.



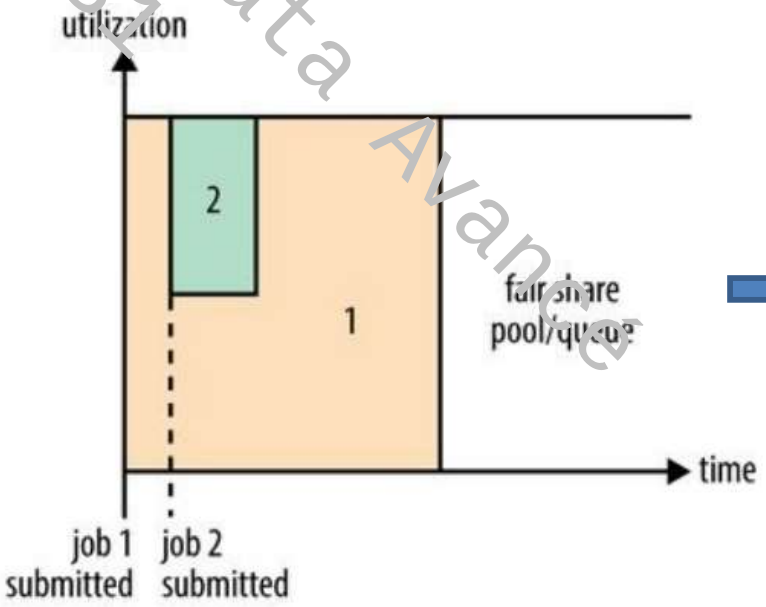
A small application (2) has to wait for the resource-hungry application (1) to finish → significant latency for User (2) despite the fact that his application does not need significant resources

- In a shared cluster, it is best to use the **capacity scheduler** or the fair scheduler. These two processes enable long-term jobs to be completed in a timely manner, while allowing users executing small simultaneous ad hoc requests to obtain results within a reasonable timeframe.



- *A separate dedicated queue allows the small job to start as soon as it is submitted. This is at the expense of overall cluster capacity as the queue capacity is reserved for the jobs in that queue.*
- *The large job finishes later than when using FIFO*

YARN:
Resources allocation: Fair



- *With **Fair allocation**, there is no need to reserve a fixed amount of capacity*
- *Fair will dynamically balance resources between all running jobs.*
- *Just after the first (large) job starts, (the only job running → it receives all the resources in the cluster)*
- *When the second (small) job starts, half the cluster resources are allocated to the second job → Job 2 runs immediately*
- *When Job 2 finishes, all cluster resources are allocated back to Job 1*

Note: Allocation Capacity and Fair require additional configurations

- Factors

- Why exactly do we need to plan a Hadoop cluster and what are the different factors we need to look at, in order to plan an efficient Hadoop cluster with optimum performance?

- **Volume of data**

It is important for a Hadoop administrator to know the volume of data they need to manage, plan, organize and implement the Hadoop cluster with the appropriate number of nodes for effective data management.

- **Data curation**

Data curation is a process where the user can remove stale, invalid and unnecessary data from Hadoop storage to save space and improve cluster compute speeds.

- Data storage

- ✓ Data storage is one of the crucial factors that come into play when planning a Hadoop cluster.
- ✓ Data is never stored directly as it is obtained. It undergoes a data compression and encryption process so that data security is ensured and the space used to store the data is as small as possible.

Hardware requirements for standard Hadoop Cluster

- The Hadoop architecture essentially comprises the following components.
  * Namenode
  * Job Tracker
  * Datanode
  * Task Tracker

- Namenode and Secondary Namenode are the crucial parts of any Hadoop Cluster. They are expected to be highly available. Namenode and Secondary Namenode servers are dedicated to namespace storage.
Example
  Octa-Core Processor 2.5 GHz
  128 GB RAM
  10 GBPS network

- Almost the same hardware requirements for DataNode and Task Tracker

- **Operating system requirements**

  - The Hadoop cluster can be configured using any operating system.

  - The most recommended systems for setting up a Hadoop cluster are
    * Solaris
    * Ubuntu
    * Fedora
    * Redhat
    * Centos

- **Planification example**

- Daily data acquisition = 100 GB
- Replication factor = 3
- Non-HDFS space = 30% (For each node, a certain amount of storage capacity is reserved for non-HDFS applications)
- DataNode hard disk size = 3TB
- Block size = 128 MB

- Number of DataNodes

  Storage space required per day = 100 Gb *3 = 300 Gb
  HDFS storage space per node = 3TB*0.7 = 2.1TB
  Number of DataNodes (per year) = (365*300GB) / 2.1TB = 51

- NameNode memory requirements
  - ✓ Depends mainly on the metadata management of the files stored in the Hadoop cluster (number of files, replication factors, etc.)
  - ✓ Generally around 1% of the data size